Introduction to Java

Outline

- Introduction
- Main characteristics of the programming language
- Variables and data types
- Input and output
- Operators
- Conditional statements
- Loops
- Arrays
- Methods (functions)
- A complete algorithm: the maximum value in an array
- Bitwise operators
- `Java`, `C++` and `Python` side by side

Introduction

Introduction: a third programming language

- In the chapters «Introduction to C++» and «Introduction to Python» we translated the pseudo-code of our algorithms into a **compiled** and into an **interpreted** programming language
- In this chapter we introduce a third programming language, Java, which is **in between**: its programs are compiled into an intermediate code, which is then executed by a virtual machine
- Its syntax is very close to the syntax of C++, but every program is organized into **classes**

The path of this chapter (1/2)

The chapter follows the same path as the two previous chapters, so that the three languages can be compared section by section:

- the main characteristics of the language and the structure of a program
- variables, data types, input and output
- operators, conditional statements and loops
- arrays, and the dynamic lists of the class `ArrayList`

The path of this chapter (2/2)

- methods (the `Java` name for functions), their parameters and their return values
- a complete algorithm, the search of the maximum value in an array, written in pseudo-code and in `Java`
- the bitwise operators and a summary table that compares `Java` with `C++` and `Python`

Every fragment of code of the chapter has been compiled with the `javac` compiler and executed with the `java` virtual machine (`OpenJDK 25`): the outputs reported in the examples are the ones printed by the programs.

Indices: pseudo-code and Java

- In pseudo-code the entries of an array $a[1..n]$ are indexed from 1 to n
- In `Java`, as in `C++` and `Python`, the elements of an array of size n are indexed from 0 to $n-1$
- The entry $a[i]$ of the pseudo-code corresponds to the element `a[i-1]` of the `Java` code

Summary

- `Java` is the third language of the course, after `C++` (compiled) and `Python` (interpreted): it is compiled into an intermediate code executed by a virtual machine
- Its syntax is close to `C++`, and every program is organized into classes
- The entry $a[i]$ of the pseudo-code is the element `a[i-1]` in `Java`

Main characteristics of the programming language

Introduction: main characteristics of the programming language

- In this section we describe the main characteristics of `Java`: how a program is executed, its data types, classes and methods
- We compare them with `C++`, with which `Java` shares most of the syntax
- Then we look at the skeleton that every `Java` program shares

Compiled and interpreted

- `Java` is a **compiled and interpreted** language:
  - the source code (`.java` files) is first compiled into **bytecode** (`.class` files) by the `Java` compiler `javac`
  - then the bytecode is executed by the **Java Virtual Machine** (JVM)

- This makes `Java` programs **platform-independent**: the same bytecode runs on any machine that has a JVM installed

"write once, run anywhere"

Types, classes and methods

- `Java` is a **strongly and statically typed** language: every variable must be declared with its data type before use, and its data type cannot change during the execution
- `Java` is **object-oriented**: every program is organized into **classes**. The entry point of every `Java` program is the `main` method of a class
- `Java` supports **methods** (the `Java` term for functions), enabling a modular and reusable approach to programming

Blocks, instructions and comments

- As in C++, the block structure of the code is defined by **curly braces** {}, and each instruction must end with the reserved symbol ";"
- As in C++, a single-line comment starts with the reserved symbol "//"
- Multi-line comments are enclosed between the reserved symbols "/*" and "*/"

Example

Comments in `Java`

This is an example of a single-line comment:

```
1 // This is a comment line
```

This is an example of a multi-line comment:

```
1 /* This is a
2    multi-line comment */
```

The structure of a program

- Before describing the single instructions, we look at the **skeleton** that every program shares
- A `Java` program is made of one or more **classes**
- The execution starts from the method

  ```
  public static void main(String[] args)
  ```

  of the class that is launched

Example (1/2)

The skeleton of a program

```
1 public class HelloWorld {
2
3     public static void main(String[] args) {
4         // The instructions of the program are written here
5         System.out.println("Hello, world!");
6     }
7 }
```

A class declared `public` must be saved in a file with the same name as the class: here `HelloWorld.java`.

Example (2/2)

The program is compiled and executed with the commands

```
javac HelloWorld.java
java HelloWorld
```

- The first command produces the bytecode file `HelloWorld.class`
- The second one executes it (note that the extension is not written)

The output is

```
Hello, world!
```

Fragments of code

- In the following examples we write only the instructions that matter
- Unless they contain a complete program, the fragments of code are meant to be placed inside the curly braces of `main`

Instructions, methods and classes

Every Java instruction belongs to a method,
and every method belongs to a class.

Each instruction ends with ";".

Summary

The language:

- compiled into bytecode by `javac`, executed by the JVM: platform-independent
- strongly and statically typed, object-oriented, organized into classes and methods
- curly braces, "`;`" and comments as in `C++`

The structure of a program:

- the execution starts from `public static void main(String[] args)`
- a `public` class is saved in a file with its name; `javac` compiles, `java` executes

Variables and data types

Introduction: variables and data types

- The source code operates on **variables**, and every variable has a **data type**
- In this section we see the fundamental data types of `Java` and how variables are declared and assigned
- We also see what happens when an integer does not fit in its data type

The fundamental data types (1/2)

The four fundamental data types of `Java` are:

1. The data type "`int`" represents an integer stored with 32 bits: a whole number from

$$-2^{31} = -2147483648 \quad \text{to} \quad 2^{31} - 1 = 2147483647$$

   For larger integers the data type `long` (64 bits) is used

2. The data type "`boolean`" represents a Boolean value: `true` or `false` (lowercase)

The fundamental data types (2/2)

3. The data type "`double`" represents a floating-point number with double precision (64 bits), as in `C++`. The data type `float` has single precision, and its values are written with the suffix `f` (e.g., `3.14f`)

4. The data type "`char`" represents a single character, enclosed in single quotes (e.g., `'A'`, `'7'`)

- The data type "`String`" (with capital `S`) represents a sequence of characters enclosed in double quotes, and it is the standard way to handle text in `Java`

Variables

- A variable acts as a container that stores a value of a specific data type for later use
- Variables are assigned values using the assignment operator "=". When a new value is assigned, the previous value is overwritten
- The compiler checks the data types: assigning a value of a wrong type (for example the string `"five"` to a variable of type `int`) or using a variable that has not received a value is a **compilation error**, and the program is not produced

Declaration before use

Variables must be declared with their data type before use, as in C++.

Example (1/2)

Variable declarations and assignments

```
1  int age = 25;                    // Integer variable
2  boolean isRaining = false;       // Boolean variable
3  double pi = 3.14159;             // Double-precision floating-point
4  float small = 3.14f;             // Single-precision floating-point
5  double large = 123456.789;       // Double-precision floating-point
6  char letter = 'A';               // Character variable
7  String name = "Alice";           // String variable
8
9  age = 26; // The previous value 25 is overwritten
10 System.out.println(age + " " + isRaining + " " + pi + " " + small);
11 System.out.println(large + " " + letter + " " + name);
```

Example (2/2)

Output:

```
26 false 3.14159 3.14
123456.789 A Alice
```

- Unlike C++, the Boolean value is displayed as `false`
- The number `123456.789` is displayed with all its digits

Example (1/2)

The limits of the data type `int`

```
1 int big = Integer.MAX_VALUE; // The largest int
2 System.out.println(big);
3 System.out.println(big + 1); // Overflow: the result does not fit in 32
      bits
4
5 long bigger = (long) big + 1; // Computed with 64 bits
6 System.out.println(bigger);
```

Output:

```
2147483647
-2147483648
2147483648
```

Example (2/2)

- When the result of an operation between `int` values does not fit in 32 bits, it "wraps around" to the other end of the interval, without any error message
- Here $2^{31} - 1$ plus 1 is printed as $-2^{31} = -2147483648$
- The integers of `Python` never overflow

Summary

Data types:

- `int` (32 bits) and `long` (64 bits), `boolean`, `double` and `float`, `char`, and `String` for text

Variables:

- declared with their data type before use, assigned with "="
- the compiler checks the data types: a wrong type is a compilation error
- an `int` that does not fit in 32 bits wraps around without any error message

Input and output

Introduction: input and output

- A program must show its results and, often, read its data
- In this section we see how to print values on the screen, also with a formatted text
- Then we see how to read input from the keyboard with the class `Scanner`

Screen output

- The object `System.out` (the standard output, provided by the class `System`) has the methods that display values on the screen
- The most common are:
  - `System.out.println()`: prints a value followed by a new line
  - `System.out.print()`: prints a value without the new line

Concatenation and formatted output

- Values and strings are **concatenated** with the + operator inside a print statement: when one of the two operands of + is a `String`, the other one is converted into text
- The expression is evaluated from left to right, so an arithmetic operation inside a concatenation must be enclosed in parentheses
- The method `System.out.printf()` prints a formatted text: in the first string,
  - `%d` is replaced by an integer, `%s` by a string
  - `%.kf` by a floating-point number with exactly k digits after the decimal point
  - `%n` by a new line

Example (1/2)

Screen output

```
1  int n = 42;
2  System.out.println("The value of the variable is " + n);
3  System.out.println("n = " + n + ", doubled = " + (n * 2));
4
5  System.out.println("n + 1 = " + n + 1);   // Concatenation: "42" then "1"
6  System.out.println("n + 1 = " + (n + 1)); // Addition, then concatenation
7
8  System.out.print("No new line ");
9  System.out.println("after print");
10 System.out.printf("%d divided by %d is %.2f%n", 10, 3, 10.0 / 3);
```

Example (2/2)

Output:

```
The value of the variable is 42
n = 42, doubled = 84
n + 1 = 421
n + 1 = 43
No new line after print
10 divided by 3 is 3.33
```

The decimal separator

- The decimal separator printed by `printf` depends on the language settings of the computer
- On a computer set to Italian the last line of the example is

  ```
  10 divided by 3 is 3,33
  ```

  with a comma
- The method `println` always uses the point

Keyboard input

- To read input from the keyboard, the class `Scanner` of the package `java.util` is used
- It must be imported with the instruction

  ```
  import java.util.Scanner;
  ```

  written at the beginning of the file, before the class
- The methods `nextLine()`, `nextInt()` and `nextDouble()` read a line of text, an integer and a floating-point number

Example (1/3)

Keyboard input with `Scanner`

```
1 import java.util.Scanner;
2
3 public class InputExample {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         System.out.print("Enter your name: ");
9         String name = sc.nextLine();       // Reads a String
```

Example (2/3)

```
11          System.out.print("Enter an integer: ");
12          int n = sc.nextInt();                  // Reads an int
13
14          System.out.print("Enter a real number: ");
15          double x = sc.nextDouble();            // Reads a double
16
17          System.out.println("Hello " + name + ", n = " + n + ", x = " +
        x);
18          sc.close();
19      }
20  }
```

Example (3/3)

Example of execution (the user types `Ada`, `7` and `2.5`):

```
Enter your name: Ada
Enter an integer: 7
Enter a real number: 2.5
Hello Ada, n = 7, x = 2.5
```

- Like `printf`, the method `nextDouble()` follows the language settings: on a computer set to Italian the real number must be typed as `2,5`

Summary

Output:

- `System.out.println()` and `System.out.print()`, with or without the new line
- concatenation with +, evaluated from left to right: arithmetic in parentheses
- `System.out.printf()` with `%d`, `%s`, `%.kf`, `%n`; its decimal separator follows the language settings

Input:

- the class `Scanner` of `java.util`, with `nextLine()`, `nextInt()` and `nextDouble()`

Operators

Introduction: operators

- Operators are symbols that perform operations on variables and values
- The operators of `Java` are the same as those of `C++`:
  - arithmetic operators
  - assignment operators
  - increment and decrement operators
  - relational operators
  - logical operators
- For each family we see the symbols and an example with its output

Arithmetic operators

Arithmetic operators, used to perform basic mathematical operations:

- + (Addition) $\rightsquigarrow$ Adds two values
- - (Subtraction) $\rightsquigarrow$ Subtracts one value from another
- * (Multiplication) $\rightsquigarrow$ Multiplies two values
- / (Division) $\rightsquigarrow$ Divides one value by another. Integer division when both operands are integers
- % (Modulus) $\rightsquigarrow$ Returns the remainder of a division

Example (1/2)

Arithmetic operators

```
1 int a = 10, b = 3;
2
3 System.out.println("Addition:          " + (a + b));
4 System.out.println("Subtraction:       " + (a - b));
5 System.out.println(" Multiplication :  " + (a * b));
6 System.out.println("Division :         " + (a / b));   // Integer division
7 System.out.println("Modulus:           " + (a % b));   // Remainder of
      division
8 System.out.println("Real division :    " + ((double) a / b));
```

Example (2/2)

Output:

```
Addition:       13
Subtraction:    7
Multiplication: 30
Division:       3
Modulus:        1
Real division:  3.3333333333333335
```

Integer division and type conversion

- As in C++, the division operator / performs **integer division** when both operands are integers, and the quotient is truncated toward zero:
  - `10 / 3` is 3
  - `-7 / 2` is `-3` (with remainder `-7 % 2` equal to `-1`)
- To obtain a floating-point result, at least one operand must be of type `double`, for example with the **type conversion** (**cast**) `(double) a / b`
- The conversion `(int)` removes the decimal part of a `double`: `(int) 3.9` is 3
- There is no operator for the power: the method `Math.pow(x, y)` returns x^y as a `double`

Example (1/2)

Integer and floating-point division

```
1 int a = 10, b = 3;
2
3 System.out.println(a / b);                    // Integer division
4 System.out.println(10.0 / 3);                 // Floating-point division
5 System.out.println((double) a / b);           // Conversion of a to double
6 System.out.printf("%.2f%n", (double) a / b);  // Two decimals
7 System.out.println(-7 / 2 + " " + -7 % 2);    // Truncation toward zero
8 System.out.println((int) 3.9 + " " + Math.pow(2, 10));
```

Example (2/2)

Output:

```
3
3.3333333333333335
3.3333333333333335
3.33
-3 -1
3 1024.0
```

Assignment operators

Assignment operators, used to assign values to variables:

- = (Assignment) $\rightsquigarrow$ Assigns a value to a variable
- += (Addition assignment) $\rightsquigarrow$ Adds a value and assigns the result
- -= (Subtraction assignment) $\rightsquigarrow$ Subtracts a value and assigns the result
- *= (Multiplication assignment) $\rightsquigarrow$ Multiplies and assigns the result
- /= (Division assignment) $\rightsquigarrow$ Divides and assigns the result
- %= (Modulus assignment) $\rightsquigarrow$ Applies the modulus and assigns the result

Example (1/2)

Assignment operators

```
1 int x = 10;  // Assignment
2 System.out.println(" Initial value: " + x);
3
4 x += 5;  System.out.println("After += 5: " + x); // x = x + 5
5 x -= 3;  System.out.println("After -= 3: " + x); // x = x - 3
6 x *= 2;  System.out.println("After *= 2: " + x); // x = x * 2
7 x /= 4;  System.out.println("After /= 4: " + x); // x = x / 4
8 x %= 4;  System.out.println("After %= 4: " + x); // x = x % 4
```

Example (2/2)

Output:

```
Initial value: 10
After += 5: 15
After -= 3: 12
After *= 2: 24
After /= 4: 6
After %= 4: 2
```

- Two instructions can be written on the same line, since each of them ends with ";"

Increment and decrement operators

Increment and decrement operators:

- ++ (Increment) $\rightsquigarrow$ Increases the value of a variable by 1
- `--` (Decrement) $\rightsquigarrow$ Decreases the value of a variable by 1

- As in `C++`, the **pre-increment** (`++x`) increments the value before returning it
- The **post-increment** (`x++`) returns the value first, then increments it
- The same logic applies to the decrement operator (`--x` and `x--`)

Example (1/2)

Increment and decrement operators

```
1 int x = 5;
2 System.out.println(" Initial value: " + x);
3 System.out.println("Pre-increment (++x): " + ++x);
4 System.out.println("Post-increment (x++): " + x++);
5 System.out.println("Value after post-increment: " + x);
6 System.out.println("Pre-decrement (--x): " + --x);
7 System.out.println("Post-decrement (x--): " + x--);
8 System.out.println("Value after post-decrement: " + x);
```

Example (2/2)

Output:

```
Initial value: 5
Pre-increment (++x): 6
Post-increment (x++): 6
Value after post-increment: 7
Pre-decrement (--x): 6
Post-decrement (x--): 6
Value after post-decrement: 5
```

Relational operators

Relational operators, used to compare values and return a `boolean` result:

- == (Equal to) $\rightsquigarrow$ Checks if two values are equal
- != (Not equal to) $\rightsquigarrow$ Checks if two values are different
- < (Less than) $\rightsquigarrow$ Checks if one value is smaller than another
- > (Greater than) $\rightsquigarrow$ Checks if one value is larger than another
- <= (Less than or equal to) $\rightsquigarrow$ Checks if one value is smaller than or equal to another
- >= (Greater than or equal to) $\rightsquigarrow$ Checks if one value is larger than or equal to another

Boolean results and strings

- Relational operators return a value of type `boolean`, displayed as `true` or `false` (not as `1` and `0` as in C++)
- These operators are commonly used in **conditional statements** and **loops**
- The operator == compares the values of the primitive data types (`int`, `double`, `char`, `boolean`)
- Two `String` values are compared with the method `equals`, for example

```
name.equals("Alice")
```

Example (1/2)

Relational operators

```
1 int a = 10, b = 5;
2
3 System.out.println("a == b: " + (a == b)); // Equal to
4 System.out.println("a != b: " + (a != b)); // Not equal to
5 System.out.println("a < b: " + (a < b));   // Less than
6 System.out.println("a > b: " + (a > b));   // Greater than
7 System.out.println("a <= b: " + (a <= b)); // Less than or equal to
8 System.out.println("a >= b: " + (a >= b)); // Greater than or equal to
```

Example (2/2)

Output:

```
a == b: false
a != b: true
a < b: false
a > b: true
a <= b: false
a >= b: true
```

Logical operators

Logical operators, used for logical operations, often in conditional statements:

- `&&` (Logical AND) $\rightsquigarrow$ Returns `true` if both conditions are `true`
- `||` (Logical OR) $\rightsquigarrow$ Returns `true` if at least one condition is `true`
- `!` (Logical NOT) $\rightsquigarrow$ Reverses the Boolean value

Example (1/2)

Logical operators

```
1 boolean c1 = true, c2 = false;
2
3 System.out.println("c1: " + c1);
4 System.out.println("c2: " + c2);
5 System.out.println("c1 && c2: " + (c1 && c2)); // True only if both are
      true
6 System.out.println("c1 || c2: " + (c1 || c2)); // True if at least one is
       true
7 System.out.println("!c1: " + (!c1));           // Reverses the value
8 System.out.println("!c2: " + (!c2));
```

Example (2/2)

Output:

```
c1: true
c2: false
c1 && c2: false
c1 || c2: true
!c1: false
!c2: true
```

Summary (1/2)

Arithmetic operators:

- `+`, `-`, `*`, `/`, `%`; between two integers `/` is the integer division, truncated toward zero
- the cast `(double)` gives a floating-point result, `(int)` removes the decimal part; `Math.pow(x, y)` for the power

Assignment, increment and decrement:

- `=`, `+=`, `-=`, `*=`, `/=`, `%=`
- `++x` increments before returning the value, `x++` after

Summary (2/2)

Relational operators:

- `==`, `!=`, `<`, `>`, `<=`, `>=` return a `boolean`, printed as `true` or `false`
- two strings are compared with `equals`

Logical operators:

- `&&`, `||`, `!`, as in C++

Conditional statements

Introduction: conditional statements

- The relational and logical operators produce the **conditions** that decide which instructions are executed
- In this section we see the statements `if`, `else if` and `else`
- Then we see the ternary operator, which chooses between two values in a single expression

The `if` statement

- The `if` statement evaluates a condition inside parentheses `()`
- If the condition is `true`, the block inside the curly braces `{}` is executed
- The optional `else if` clause checks additional conditions, and the optional `else` clause handles all the remaining cases
- Unlike C++, the condition must be of type `boolean`: an integer is not accepted as a condition
  - `if (n % 2)` is a compilation error
  - the test must be written `if (n % 2 != 0)`

Example

Conditional statement

```
1 int number = 10;
2
3 // Check if number is positive
4 if (number > 0) {
5     System.out.println("The number is positive.");
6 } else {
7     System.out.println("The number is non-positive.");
8 }
```

Output:

```
The number is positive.
```

Example (1/2)

`if / else if / else`

```
1  int score = 85;
2  String grade;
3
4  if      (score >= 90) { grade = "A"; }
5  else if (score >= 80) { grade = "B"; }
6  else if (score >= 70) { grade = "C"; }
7  else if (score >= 60) { grade = "D"; }
8  else                  { grade = "F"; }
9
10 System.out.println ("Score: " + score + ", Grade: " + grade);
```

Output:

```
Score: 85, Grade: B
```

Example (2/2)

- The conditions are evaluated from top to bottom, and only the block of the first true condition is executed
- The variable `grade` is declared without a value: the compiler accepts it because every branch assigns a value to it before it is printed

Ternary operator (conditional operator)

- The **ternary operator** `?:` is the same as in C++
- Syntax:

```
condition ? value_if_true : value_if_false
```

- The expression takes the first value when the condition is `true`, the second one otherwise

Example (1/3)

Ternary operator

```
1  int number = 10;
2
3  // condition ? value_if_true : value_if_false
4  String result = (number > 0) ? "positive" : "non-positive";
5  System.out.println("The number is " + result);
6
7  // Finding the maximum of two numbers
8  int a = 15, b = 20;
9  int maximum = (a > b) ? a : b;
10 System.out.println("Maximum: " + maximum);
```

Example (2/3)

```
12 // Checking even or odd
13 int n = 7;
14 String parity = (n % 2 == 0) ? "even" : "odd";
15 System.out.println(n + " is " + parity);
16
17 // Absolute value
18 int x = -5;
19 int absValue = (x >= 0) ? x : -x;
20 System.out.println("Absolute value of " + x + " is " + absValue);
```

Example (3/3)

Output:

```
The number is positive
Maximum: 20
7 is odd
Absolute value of -5 is 5
```

- The class `Math` also provides the methods `Math.max(a, b)` and `Math.abs(x)`, which return the same values

Summary

Conditional statements:

- `if`, `else if`, `else`: only the block of the first true condition is executed
- the condition must be of type `boolean`: `if (n % 2 != 0)`, not `if (n % 2)`

Ternary operator:

- `condition ? value_if_true : value_if_false`, as in C++
- `Math.max` and `Math.abs` give the maximum and the absolute value

Loops

Introduction: loops

- Conditional statements choose *whether* a block of code is executed; loops decide *how many times* it is executed
- Loops are used to execute a block of code repeatedly while a condition is met
- As in `C++`, the most commonly used loops of `Java` are the `while` loop and the `for` loop; we also see `break` and `continue`

The `while` loop

- The `while` loop executes a block of code as long as the condition inside the parentheses `()` evaluates to `true`. When the condition is `false`, the loop terminates

Example

```
1 int i = 1; // Initialization
2
3 while (i <= 5) { // Condition
4     System.out.println("Number: " + i);
5     i++; // Increment
6 }
```

Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

The `for` loop

- The `for` loop combines initialization, condition, and increment (or decrement) in one statement, exactly as in C++

Example

```
1 //  Initialization  ; Condition; Increment
2 for (int i = 1; i <= 5; i++) {
3     System.out. println ("Number: " + i);
4 }
```

The output is the same as in the `while` loop of the previous example.

The `break` and `continue` statements

- The `break` statement immediately terminates the loop
- The `continue` statement skips the rest of the current iteration and moves to the next one

Example

`break` **and** `continue`

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // Skip the even numbers
    }
    if (i > 7) {
        break; // Stop the loop at the first odd number greater than 7
    }
    System.out.print(i + " ");
}
System.out.println();
```

Output:

```
1 3 5 7
```

The output is the same as in the `C++` and `Python` versions: the value 9 triggers the `break`.

Summary

Loops:

- `while`: repeats a block as long as the condition is `true`
- `for`: initialization, condition and increment in one statement, as in C++

Jumps:

- `break` terminates the loop
- `continue` moves to the next iteration

Arrays

Introduction: arrays

- Loops become really useful when they visit **collections** of values
- A collection of variables of the same data type is stored in an **array**
- In this section we see one- and two-dimensional arrays, and the lists of variable size of the class `ArrayList`
- Arrays are the input of almost all the algorithms of the course

Arrays in Java (1/2)

- An **array** in `Java` is an ordered collection of elements of the same data type, with a size fixed at creation time: the size cannot change afterwards
- The data type of an array of integers is written `int[]`
- An array is allocated with the `new` operator, for example `new int[n]`, where the size `n` can be decided during the execution, as for the dynamic-size arrays of `C++`
- The elements of a new array are initialized to `0` (to `false` for `boolean`)

Arrays in Java (2/2)

- Elements are accessed using their **index** inside square brackets `[]`, starting from `0`: the elements of an array of size n are

  `a[0]`, `a[1]`, ..., `a[n-1]`

- Every array knows its size: the length of an array `a` is `a.length`

- Unlike `C++`, the memory of an array must not be freed with `delete[]`: it is released automatically by the **garbage collector** of the JVM when the array is no longer used

Arrays: the essentials

Arrays are allocated with `new` before use
(or created with a list of values in curly braces).

Indices start at `0`,
the size is fixed and is given by `a.length`.

Example (1/2)

Creating and accessing an array

```
1  int n = 5;
2
3  int [] a = new int[n]; // Allocation of an array of n integers
4
5  for (int i = 0; i < n; i++) {
6      a[i] = i * 2; // Example of value assignment
7  }
8
9  System.out.println("Length: " + a.length);
10 System.out.println("a[0] = " + a[0]); // First element
11 System.out.println("a[4] = " + a[4]); // Last element, index n-1
```

Output:

```
Length: 5
a[0] = 0
a[4] = 8
```

Example (2/2)

- Accessing an index that does not exist, for example `a[5]`, stops the program with the error

```
ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 5
```

- In C++ the same access is not checked

Array literals and the enhanced `for` loop

- An array whose values are known when the program is written can be created by listing its values inside curly braces, as in `C++`
- The **enhanced `for` loop** `for (int x : a)` visits the elements of the array `a` directly, without using their indices, like the `for` loop over a list of `Python`
- The method `Arrays.toString(a)`, of the class `Arrays` of the package `java.util`, returns the elements of an array as a text in square brackets

Example (1/2)

Array literal and iteration

```
1  import java.util.Arrays;
2
3  int[] a = {9, 12, 3, 16, 19, 11, 12};
4  int n = a.length;
5
6  for (int i = 0; i < n; i++) {       // Loop over the indices
7      System.out.println("a[" + i + "] = " + a[i]);
8  }
9
10 for (int x : a) {                   // Loop over the elements
11     System.out.print(x + " ");
12 }
13 System.out.println();
14
15 System.out.println(Arrays.toString(a));
```

Example (2/2)

Output:

```
a[0] = 9
a[1] = 12
a[2] = 3
a[3] = 16
a[4] = 19
a[5] = 11
a[6] = 12
9 12 3 16 19 11 12
[9, 12, 3, 16, 19, 11, 12]
```

The `import` instruction is written at the beginning of the file, before the class.

Two-dimensional arrays (1/2)

- Two-dimensional arrays (matrices) are declared with two pairs of square brackets `[][]`
- They are allocated with `new int[m][n]` (m rows and n columns) in a single instruction
- The element in row i and column j is accessed as `matrix[i][j]`, with both indices starting from 0
- `matrix.length` is the number of rows and `matrix[i].length` is the number of columns of row i

Two-dimensional arrays (2/2)

- A two-dimensional array can also be created by listing its values row by row:

```
int[][] c = {{3, 1, 4}, {1, 5, 9}};
```

  creates an array with 2 rows and 3 columns

- `c[1][2]` is the element in the second row and third column, equal to 9

Example (1/2)

Two-dimensional array (matrix)

```
1  int m = 3; // Number of rows
2  int n = 2; // Number of columns
3
4  int [][] matrix = new int[m][n]; // Allocation of an m x n matrix
5
6  for (int i = 0; i < m; i++) {
7      for (int j = 0; j < n; j++) {
8          matrix[i][j] = i * j; // Example of value assignment
9      }
10 }
11
12 for (int i = 0; i < m; i++) {
13     for (int j = 0; j < n; j++) {
14         System.out.println("matrix[" + i + "][" + j + "] = " + matrix[i][
       j]);
15     }
16 }
```

Example (2/2)

Output:

```
matrix[0][0] = 0
matrix[0][1] = 0
matrix[1][0] = 0
matrix[1][1] = 1
matrix[2][0] = 0
matrix[2][1] = 2
```

Lists of variable size: `ArrayList`

- When the number of elements is not known in advance, the class `ArrayList` of the package `java.util` provides a list that grows during the execution, like the lists of `Python`
- The data type of a list of integers is `ArrayList<Integer>`: the elements of an `ArrayList` are objects, and `Integer` is the class that corresponds to `int`
- The method `add(x)` appends the element `x` at the end of the list, `get(i)` returns the element of index i, `set(i, x)` replaces it, and `size()` returns the number of elements

Example

A list of variable size

```
1  import java.util.ArrayList;
2
3  ArrayList<Integer> list = new ArrayList<>(); // Empty list
4
5  for (int i = 1; i <= 4; i++) {
6      list.add(i * i);   // The list grows
7  }
8  list.set(0, 100);      // Replaces the element of index 0
9
10 System.out.println( list + " has " + list.size() + " elements");
11 System.out.println(" list.get(2) = " + list.get(2));
```

Output:

```
[100, 4, 9, 16] has 4 elements
list.get(2) = 9
```

Summary

Arrays:

- `int[] a = new int[n];` or a list of values in curly braces; the size is fixed
- indices from `0` to `a.length - 1`; an index out of range stops the program with an exception
- the enhanced `for` loop and `Arrays.toString(a)`; memory freed by the garbage collector

Matrices and lists:

- `new int[m][n]`, element `matrix[i][j]`
- `ArrayList<Integer>` grows with `add`; `get`, `set`, `size`

Methods (functions)

Introduction: methods (functions)

- As in pseudo-code, an algorithm is written as a function that receives the input and returns the output
- In `Java`, functions are called **methods** and must always be defined inside a **class**
- In this section we see how a method is written and called, and how its parameters and return values work

The parts of a method

A method in `Java` consists of:

1. an optional **access modifier** (e.g., `public`, `private`) and the keyword `static` when the method belongs to the class rather than to an object of the class (all the methods of this chapter are `static`)
2. a **return type**: specifies the type of the returned value (`void` if nothing is returned)
3. a **method name**: the unique identifier of the method
4. a **parameter list** (optional): specifies the input values, each one with its data type
5. a **body**: the block of code executed when the method is called
6. a `return` statement, mandatory if the return type is not `void`

The entry point and the order of the methods

- The **entry point** of every `Java` program is the method

  ```
  public static void main(String[] args)
  ```

  which plays the role of the `main` function of `C++`
- The parameter `args` contains the words written after the name of the class in the command `java`

- Unlike `C++`, a method can be called before the point where it is defined in the class: the order of the methods inside a class does not matter

- `Java` source code can also be compiled and executed online, for example at
  https://www.mycompiler.io/online-java-compiler

Example (1/2)

Methods

```
1  public class Main {
2
3      // Method definition
4      static int addition(int a, int b) {
5          return a + b; // Returns the sum
6      }
7
8      // Method definition
9      static int multiplication (int a, int b) {
10         return a * b; // Returns the product
11     }
```

Example (2/2)

```
13      public static void main(String[] args) {
14          int sum = addition(5, 10);              // Method call
15          System.out.println("Sum: " + sum);
16
17          int prod = multiplication(5, 10);   // Method call
18          System.out.println("Prod: " + prod);
19      }
20  }
```

Output:

```
Sum: 15
Prod: 50
```

Example (1/2)

Method operating on a two-dimensional array

```
1  public class Main {
2
3      static int sumMatrix(int [][] matrix, int rows, int cols) {
4          int sum = 0;
5          for (int i = 0; i < rows; i++) {
6              for (int j = 0; j < cols; j++) {
7                  sum += matrix[i][j];
8              }
9          }
10         return sum;
11     }
```

Example (2/2)

```
13      public static void main(String[] args) {
14          int m = 3, n = 2;
15          int[][] matrix = new int[m][n];
16
17          for (int i = 0; i < m; i++)
18              for (int j = 0; j < n; j++)
19                  matrix[i][j] = i * j;
20
21          int total = sumMatrix(matrix, m, n);
22          System.out.println("Sum of matrix elements: " + total);
23      }
24  }
```

Output:

```
Sum of matrix elements: 3
```

The body of a `for` loop made of a single instruction can be written without curly braces, as in C++.

Parameters and return values (1/2)

- In `Java` every parameter is passed **by value**: the method receives a copy of the argument
- For the primitive data types (`int`, `double`, `boolean`, `char`) the changes made to the copy do not affect the variable of the caller
- There is no passing by reference as with the symbol `&` of `C++`

Parameters and return values (2/2)

- A variable of type `int[]` does not contain the elements, but a **reference** to the array (similar to the pointer of `C++`)
- The method receives a copy of the reference, which points to the same array: the changes made to the *elements* of the array are visible to the caller
- A method returns at most one value. When a method has to give back several results, it can return them in an array, for example `new int[]{q, r}`

Example (1/3)

Parameters and multiple results

```
1  public class Main {
2
3      static void increment(int x) {
4          x = x + 1; // Modifies only the local copy
5      }
6
7      static void fillWithZeros(int[] a) {
8          for (int i = 0; i < a.length; i++) {
9              a[i] = 0; // Modifies the array of the caller
10         }
11     }
12
13     static int[] divide(int n, int m) {
14         return new int[]{n / m, n % m}; // Two results in an array
15     }
```

Example (2/3)

```
17      public static void main(String[] args) {
18          int k = 5;
19          increment(k);
20          System.out.println("After increment: " + k);
21
22          int[] b = {4, 7, 1};
23          fillWithZeros(b);
24          System.out.println("Array: " + b[0] + " " + b[1] + " " + b[2]);
25
26          int[] qr = divide(17, 5);
27          System.out.println("Quotient and remainder: " + qr[0] + " " + qr
        [1]);
28      }
29  }
```

Example (3/3)

Output:

```
After increment: 5
Array: 0 0 0
Quotient and remainder: 3 2
```

- `increment` changes only its copy: `k` is still 5
- `fillWithZeros` receives a copy of the reference to `b`, and sets to 0 the elements of the same array
- `divide` returns quotient and remainder of 17 divided by 5 in an array

Summary

Methods:

- defined inside a class: modifiers, return type, name, parameters, body, `return`
- the entry point is `public static void main(String[] args)`; the order of the methods does not matter

Parameters and return values:

- every parameter is passed by value; an array is passed through a copy of its reference, so its elements can be modified
- several results are returned together in an array

A complete algorithm: the maximum value in an array

Introduction: a complete algorithm

- We now put all the pieces together: a method, a loop, a conditional statement and an array
- We translate the pseudo-code of an algorithm into `Java`, taking care of the indices
- Then we test it on a fixed instance and on a random one, and we follow its execution step by step

The problem

We consider the problem of finding the maximum value in an array a of $n \geq 1$ integer numbers.

- The next slide shows the pseudo-code of the algorithm `FindMaximum` of the chapter «Pseudo-code»: the entries are indexed from 1 to n

The pseudo-code

Algorithm: FindMaximum

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output: The maximum value in the array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

From pseudo-code to Java

- In the `Java` implementation the elements are indexed from 0 to $n - 1$, so the initialization uses `a[0]` and the loop starts from `i = 1`
- The size n is not passed as a parameter, because the method obtains it with `a.length`
- The method assumes $n \geq 1$, as the pseudo-code does: for an empty array the access to `a[0]` stops the program with an `ArrayIndexOutOfBoundsException`

The Java implementation (1/2)

```
1  public class Main {
2
3      // Method to find the maximum value in an array
4      static int arrayMaxValue(int[] a) {
5          int max = a[0]; // Initialize with the first element
6
7          for (int i = 1; i < a.length; i++) {
8              if (a[i] > max) {
9                  max = a[i]; // Update the maximum
10             }
11         }
12
13         return max;
14     }
```

The Java implementation (2/2)

```
16      public  static  void  main(String []  args) {
17          int []  a = {9, 12,  3,  16,  19,  11,  12};
18
19          int  max = arrayMaxValue(a);
20          System.out. println ("Maximum value: " + max);
21      }
22  }
```

Output:

```
Maximum value: 19
```

A random instance

- The test instance can also be generated at random with the class `Random` of the package `java.util`
- If `rand` is an object of this class, `rand.nextInt(100)` returns a random integer between 0 and 99, so `rand.nextInt(100) + 1` is between 1 and 100
- The object created with `new Random(2026)` starts from the fixed value 2026 (the **seed**), so that every execution produces the same array
- The object created with `new Random()` produces a different array at every execution

Example (1/2)

A random instance

```
1  import java.util.Arrays;
2  import java.util.Random;
3
4  public class Main {
5
6      static int arrayMaxValue(int[] a) {
7          int max = a[0];
8          for (int i = 1; i < a.length; i++) {
9              if (a[i] > max) {
10                 max = a[i];
11             }
12         }
13         return max;
14     }
```

Example (2/2)

```
16      public static void main(String[] args) {
17          int n = 10; // Size of the array
18          int[] a = new int[n];
19          Random rand = new Random(2026); // Same random array at every
        execution
20          for (int i = 0; i < n; i++) {
21              a[i] = rand.nextInt(100) + 1; // Random integer between 1
        and 100
22          }
23
24          System.out.println(Arrays.toString(a));
25          System.out.println("Maximum value: " + arrayMaxValue(a));
26      }
27  }
```

Output:

```
[100, 51, 98, 31, 50, 39, 74, 37, 35, 79]
Maximum value: 100
```

Example (1/3)

Step-by-step execution

We follow the execution of `arrayMaxValue` on two instances, with a table for each one:

- the column i is the index of the `Java` loop
- the column $i + 1$ is the corresponding index of the pseudo-code
- the first row is the initialization `max = a[0]`

Example (2/3)

Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i (Java)	$i + 1$ (pseudo-code)	a[i]	a[i] > max?	max
–	–	–	–	9
1	2	12	Yes	12
2	3	3	No	12
3	4	16	Yes	16
4	5	19	Yes	19
5	6	11	No	19
6	7	12	No	19

Return: max $= 19$

Example (3/3)

Instance 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i (Java)	$i + 1$ (pseudo-code)	a[i]	a[i] > max?	max
–	–	–	–	3
1	2	7	Yes	7
2	3	2	No	7
3	4	9	Yes	9
4	5	1	No	9

Return: max $= 9$

Summary

From pseudo-code to `Java`:

- $a[1]$ becomes `a[0]`, the loop from 2 to n becomes the loop from `1` to `a.length - 1`
- the size is obtained with `a.length`; the method assumes $n \geq 1$

Testing:

- on a fixed array, and on a random one generated with `Random`: with a seed every execution produces the same array
- the step-by-step tables follow the index i of `Java` and $i + 1$ of the pseudo-code

Bitwise operators

Introduction: bitwise operators

- The bitwise operators work on the binary representation of the integers
- They exist in `Java` as in `C++`: in this section we see the few differences, which depend on the 32 bits of `int` and on the conditions of type `boolean`

The same operators as in C++

- The bitwise operators of the chapter «Introduction to C++» exist in Java with the same symbols and the same meaning:
  - & (AND), | (OR), ^ (XOR), ~ (NOT)
  - << (left shift) and >> (right shift)

- The method `Integer.toBinaryString(x)` returns the binary representation of an integer

What changes in Java

- A value of type `int` has 32 bits, so `~12` flips 32 bits and gives `-13`; the 8-bit result of `C++` is obtained with the mask `0xFF`, as in `Python`
- `Java` has a third shift operator, `>>>` (unsigned right shift), which always adds zeros on the left, also for negative numbers; for non-negative numbers it coincides with `>>`
- Since a condition must be of type `boolean`, the test of the least significant bit is written `(n & 1) == 1`, and not `(n & 1)` as in `C++`

Example (1/2)

Bitwise operators

```
1  int a = 12; // Binary: 1100
2  int b = 10; // Binary: 1010
3
4  System.out.println((a & b) + " " + (a | b) + " " + (a ^ b)); // AND, OR,
       XOR
5  System.out.println(Integer.toBinaryString(a & b));              // 1000
6  System.out.println((5 << 2) + " " + (20 >> 2) + " " + (20 >> 3)); //
       Shifts
7  System.out.println(~a + " " + (~a & 0xFF));  // NOT, and its 8-bit
       version
8  System.out.println((-8 >> 1) + " " + (-8 >>> 28)); // Signed and
       unsigned shift
9
10 int n = 15;
11 System.out.println(((n & 1) == 1) ? "odd" : "even"); // Least significant
       bit
```

Example (2/2)

Output:

```
8 14 6
1000
20 5 2
-13 243
-4 15
odd
```

- The number -8 is stored as $11\ldots11000$ (32 bits)
- `-8 >> 1` keeps the sign and gives -4
- `-8 >>> 28` keeps only the 4 leftmost bits 1111, that is 15

Summary

Bitwise operators:

- `&`, `|`, `^`, `~`, `<<`, `>>` as in `C++`; `Integer.toBinaryString(x)` shows the bits
- an `int` has 32 bits: `~12` is `-13`, and the mask `0xFF` gives the 8-bit result

Only in `Java`:

- the unsigned right shift `>>>`, which adds zeros on the left
- the test of the least significant bit is `(n & 1) == 1`

Java, C++ and Python side by side

Introduction: Java, C++ and Python side by side

- We have now seen the three languages of this part of the course
- The following table summarizes the main differences among them, row by row
- It is a useful reference when an algorithm written in pseudo-code has to be translated into one of the three languages

Comparison table (1/3)

	C++	Java	Python
Execution	compiled (g++)	compiled into bytecode (`javac`), executed by the JVM (`java`)	interpreted (`python3`)
Data types	declared	declared	not declared
Blocks	`{}`	`{}`	`:` and indentation
End of instruction	;	;	end of the line

Comparison table (2/3)

	C++	Java	Python
Boolean values	`true`, `false` (printed `1`, `0`)	`true`, `false`	`True`, `False`
`10 / 3`	`3`	`3`	`3.3333333333333335`
`-7 / 2` (integer)	`-3`	`-3`	`-7 // 2` is `-4`
Largest integer	$2^{31} - 1$ for `int`	$2^{31} - 1$ for `int`	no limit
Logical operators	`&&`, `||`, `!`	`&&`, `||`, `!`	`and`, `or`, `not`

Comparison table (3/3)

	C++	Java	Python
Collections	arrays, freed with `delete[]`	arrays (`a.length`) and `ArrayList`, freed automatically	lists (`len(a)`), freed automatically
Out-of-range index	not checked	exception	exception
Several results	parameters by reference	return of an array	return of a tuple
Entry point	function `main`	method `main` of a class	top level of the file

Summary

- `Java` shares with `C++` the declared data types, the blocks in curly braces, the "`;`", the integer division and the 32-bit `int`
- `Java` shares with `Python` the checked indices and the automatic release of the memory
- Only `Java` needs a class around every program, and returns several results in an array

Summary (1/2)

Key points of the chapter

- A `Java` program is made of classes; it is compiled into bytecode with `javac` and executed by the JVM with `java`, starting from the method `public static void main(String[] args)`

- Every variable is declared with its data type, which is checked by the compiler; the data type `int` has 32 bits, and the division between two integers is the integer division

- Conditional statements (`if`, `else if`, `else`, `?:`) and loops (`while`, `for`, and the enhanced `for`) have the same syntax as in `C++`, but a condition must be of type `boolean`

Summary (2/2)

Key points of the chapter

- The arrays of `Java` are allocated with `new`, know their size `a.length`, are indexed from 0 to $n-1$ and are freed automatically; the class `ArrayList` provides lists of variable size

- Parameters are passed by value; a method that receives an array can modify its elements, and several results are returned together in an array